Python Fundamentals

Condition and Branching Statements, Built- in data structures: List, Tuple, Dictionary, Set

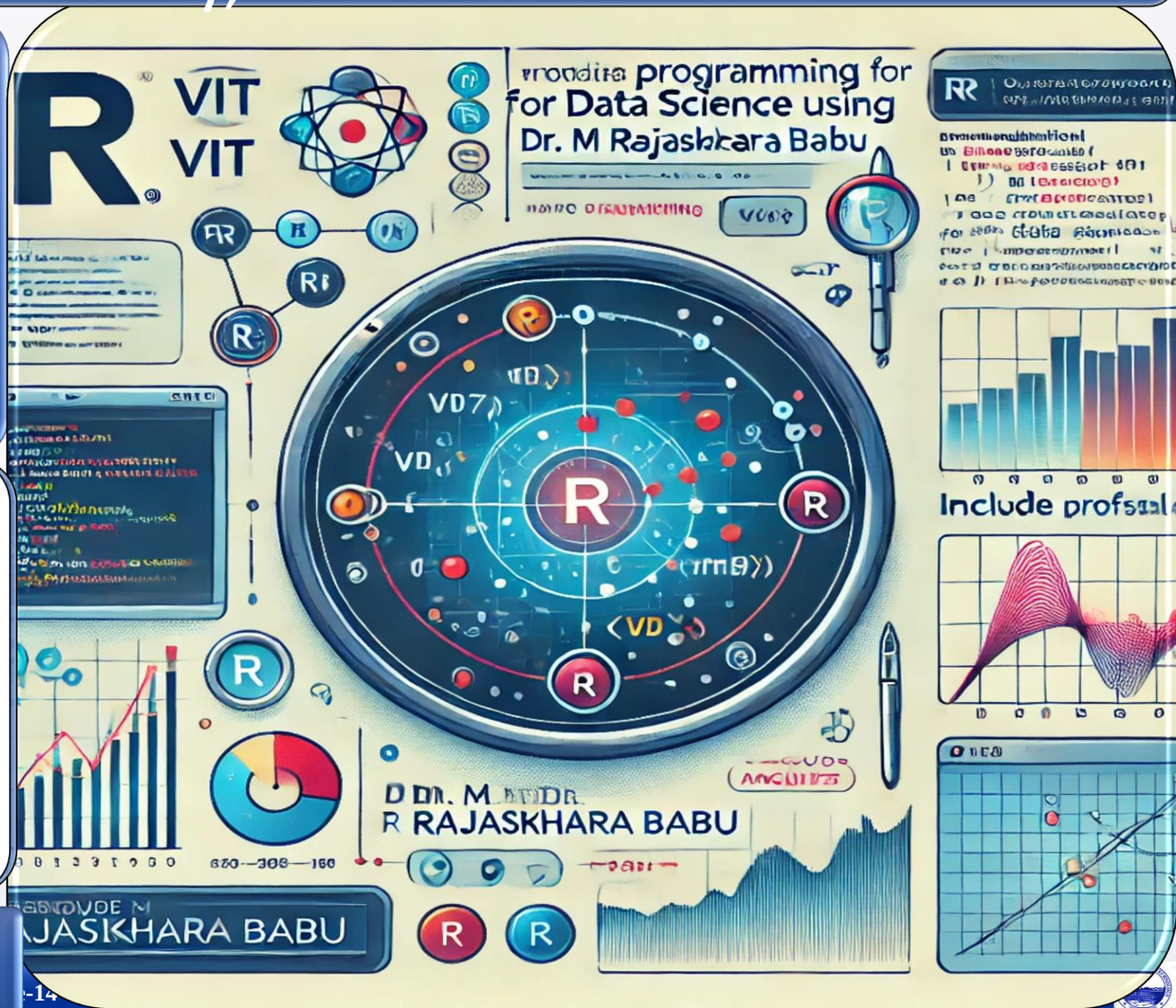
by

Dr M Rajasekhara Babu

School of

Computer Science and Engineering

https://www.rajababu.org



VELLORE INSTITUTE OF TECHNOLOGY
VIT
உழைப்பே உயர்வு தரும்

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India

Objectives & Teaching Learning Material

To explain the fundamental concepts of Python programming, including variables, data types, operators, and input/output operations.

To demonstrate the use of Python built-in data structures, conditional statements, loops, and functions for problem solving.

To illustrate how Python fundamentals support data analysis tasks and serve as a foundation for working with Pandas DataFrames.

Teaching + Learning Material: LCD, White board Marker, Presentation slides

Learning outcomes

Upon successful completion of this lecture, students will be able to:

Explain the fundamental concepts of Python programming, including variables, data types, operators, and input/output operations

Apply Python built-in data structures, conditional statements, loops, and functions to develop simple programs and solve computational problems

Construct and manipulate data using lists and dictionaries, and relate these structures to DataFrame creation for data analysis applications

Session Plan

Time in minutes	Topic	Learning Aid & Methodology	Faculty Approach	Student Activity	Skill and Competency Developed
05	Re-Cap	Quiz Presentation	Questions Organizes	Answers Identifies	Knowledge Understanding
10	Python Basics: Variables, Data Types, Operators, and I/O	Presentation	Presentation	Listens	Remembering
12	Built-in Data Structures: Lists and Dictionaries	Explains	Explains	Notes	Understanding
13	Control Flow: Conditional Statements and Loops	Case Study	Organizes	Observes	Applying
10	Functions and Python for Data Analysis	Discussion	Facilitates	Answers	Analyzing

Learning Objectives

After this session, students will be able to:

Core Python

Understand Python basics, variables, and data types

Data Collections

Use lists and dictionaries to store and access data

Write Programs

Write simple Python programs from scratch

Pandas Ready

Prepare data structures for Pandas DataFrames

Introduction to Python

What is Python?

- High-level programming language
- Interpreted language
- Open-source and easy to learn

Applications

- Data Science & Machine Learning
- Artificial Intelligence
- Web Development & Automation
- Scientific Computing

```
print("Hello World")
# Output: Hello World
```

Variables in Python

A variable is a named memory location used to store data. Syntax:

```
variable_name = value
```

Examples

```
name = "Alice"
age = 23
salary = 50000
student_name = "John"
marks = 90
```

Naming Rules

- Can contain letters, digits, underscore
- Cannot start with a number
- Case sensitive

Data Types in Python

Python supports various built-in data types.
Use `print(type(age))` to check — output: `<class 'int'>`

int
Whole numbers

```
age = 23
```

float
Decimal numbers

```
height = 5.6
```

str
Text strings

```
name = "Alice"
```

bool
True or False

```
is_student = True
```

Operators in Python

Arithmetic operators are used in calculations and data preprocessing.

Code Example

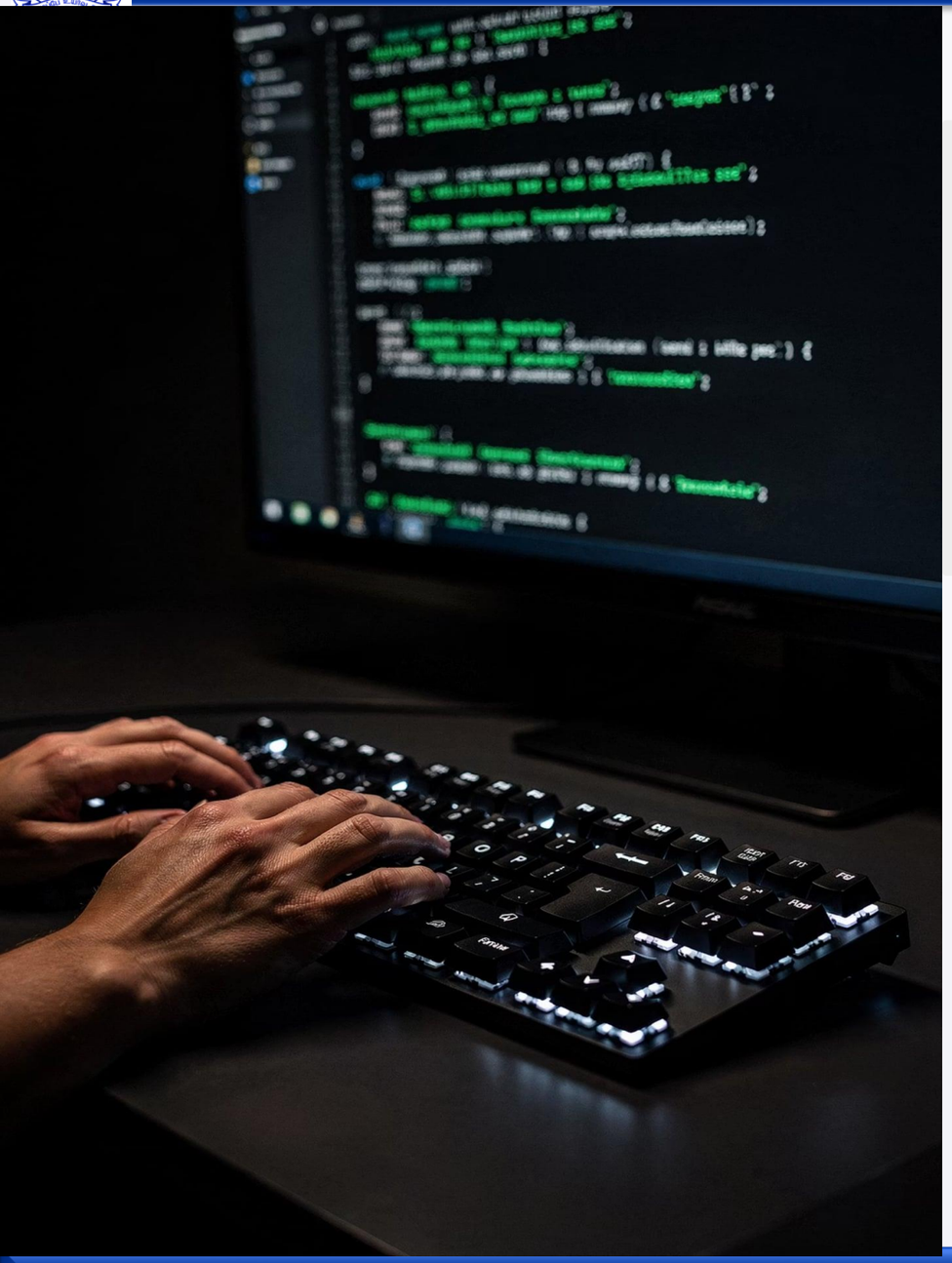
```
a = 10
b = 5

a + b   # 15
a - b   # 5
a * b   # 50
a / b   # 2.0
a % b   # 0
```

Operator Summary

- + Addition
- - Subtraction
- * Multiplication
- / Division
- % Modulus (remainder)

Input and Output



Taking Input

```
name = input("Enter name:")
```

Displaying Output

```
print(name)
```

Example with Output

```
age = 23
print("Age =", age)

# Output:
# Age = 23
```

Python Lists

A list stores multiple values in a single variable. Syntax: `list_name = [value1, value2, value3]`

Example

```
ages = [23, 30, 28, 35, 22]
names = ["Alice", "Bob", "Charlie"]
```

Characteristics

Ordered

Elements maintain their position

Mutable

Values can be changed

Duplicates

Allows repeated values

Accessing List Elements

List elements are accessed using their **index position**, starting at `0`.

Index Positions

Element	Index
Alice	0
Bob	1
Charlie	2

Access Elements

```
names = ["Alice", "Bob", "Charlie"]

print(names[0]) # Alice

print(names[1]) # Bob
```

List Operations

Lists support a variety of operations — used frequently when creating datasets.

Add Elements

```
names.append("David")
```

Update Elements

```
names[1] = "John"
```

Length of List

```
len(names)

marks = [70, 80, 90]
```

Python Dictionaries

A dictionary is a collection of key-value pairs.

Syntax & Example

```
data = {
  "Name": "Alice",
  "Age": 23
}
```

Structure

Key	Value
Name	Alice
Age	23

Accessing & Updating Dictionary Values

Access Value

```
student = {
  "Name": "Alice",
  "Age": 23
}

print(student["Name"])
# Output: Alice
```

Update Value

```
student["Age"] = 24
```

Dictionary values are mutable — you can update any value by referencing its key directly.

Dictionary for Pandas DataFrame

Dictionaries are the primary structure used to create Pandas DataFrames.

Data Structure

```
data = {
  "Name": ["Alice","Bob","Charlie"],
  "Age":  [23, 30, 28],
  "City": ["New York","London","Paris"]
}
```

Why Dictionary?

- **Keys → Column Names**
- **Lists → Column Values**
- **Easily converted into a DataFrame**

Conditional Statements

The `if` statement executes code when a condition is True. Widely used for filtering data in analysis.

Syntax

```
if condition:
    statement
```

Example

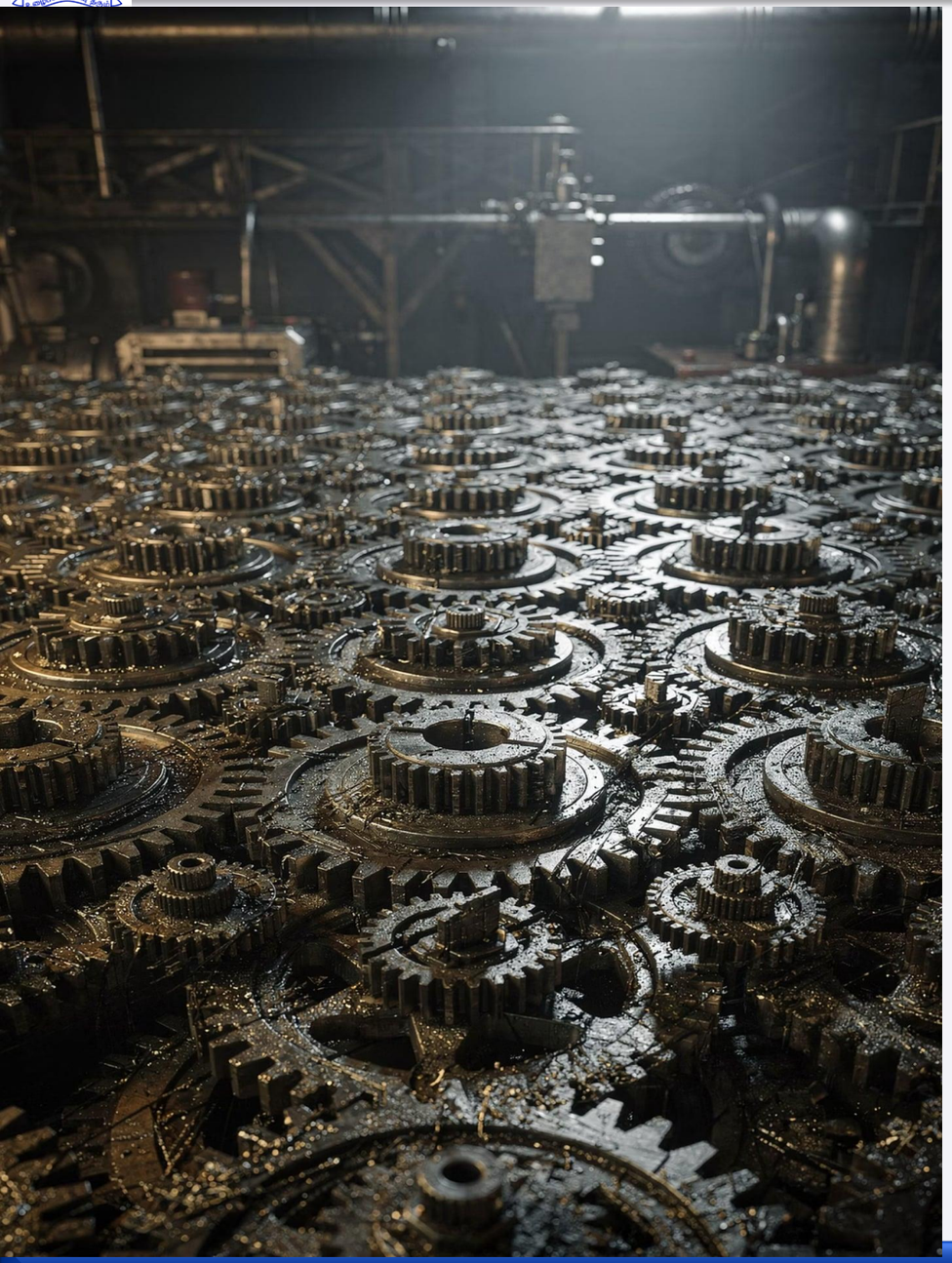
```
age = 30
if age > 25:
    print("Eligible")

# Output: Eligible
```

Comparison Operators

Operator	Meaning
>	Greater Than
<	Less Than
>=	Greater Than or Equal
<=	Less Than or Equal
==	Equal
!=	Not Equal

Loops in Python



A for loop repeats an action for each item in a collection — essential for processing datasets.

Example

```
names = ["Alice", "Bob", "Charlie"]

for name in names:
    print(name)

# Output:
# Alice
# Bob
# Charlie
```

Why Loops Matter

- Iterate over list elements
- Automate repetitive tasks
- Process rows in a dataset
- Core tool in data operations

Functions in Python

A function is a reusable block of code defined with the `def` keyword. Functions keep code organized and avoid repetition.

Define & Call

```
def greet():
    print("Welcome")

greet()
# Output: Welcome
```

Student Data Example

```
name = "Alice"
age = 23
city = "New York"

print(name) # Alice
print(age) # 23
print(city) # New York
```

Connecting Python to Pandas

Every Python concept covered in this module maps directly to a Pandas operation.

Python Concept	Used In Pandas
Variables	Store Data
Lists	Column Values
Dictionaries	Create DataFrame
Operators	Filtering
Functions	Data Operations
Loops	Data Processing

```
data = {"Name": ["Alice", "Bob"], "Age": [23, 30]}
df = pd.DataFrame(data)
```

Practice Exercise

Write a Python program that demonstrates all the fundamentals covered in this module.

01

Create Variables

Define variables for `Name` and `Age`

02

Build a List

Create a list of five marks

03

Build a Dictionary

Create a dictionary with `Name`, `Age`, and `City`

04

Display Values

Print all values using `print()`

05

Conditional Check

Check whether `Age` is greater than 25

Expected Outcome: Students will be prepared to learn and implement DataFrames using Pandas.

Summary: Topics Covered

Core Concepts
- Python Introduction
- Variables & Data Types
- Input / Output

Data Collections
- Lists
- Dictionaries
- Operators

Control Flow
- Conditional Statements
- Loops
- Functions

Pandas Bridge
- Preparing Data
- DataFrame Creation

Key Takeaway

Python Fundamentals provide the foundation for learning Pandas, Data Analysis, Machine Learning, and Data Science applications.

1. **Python Basics**
   Variables, types, operators
2. **Data Structures**
   Lists & dictionaries
3. **Pandas & Beyond**
   DataFrames, ML, AI

Summary

- **Introduction to Python**
  - Features and applications of Python
  - Python as a high-level, interpreted programming language
- **Variables and Data Types**
  - Variable declaration and naming rules
  - Built-in data types: int, float, str, and bool
- **Operators and Input/Output**
  - Arithmetic operators
  - User input using input()
  - Output using print()
- **Built-in Data Structures**
  - Lists
  - List operations and indexing
  - Dictionaries
  - Accessing and updating dictionary elements
  - Dictionaries for DataFrame creation
- **Conditional and Branching Statements**
  - if statements
  - Comparison operators
  - Decision-making in Python
- **Loops in Python**
  - for loops
  - Iterating through collections
  - Processing data using loops
- **Functions in Python**
  - Defining functions using def
  - Function calls and code reusability
- **Connecting Python Fundamentals to Pandas**
  - Using lists and dictionaries for DataFrame creation
- Role of Python fundamentals in data analysis

Reference

Alvaro Fuentes, Become a Python Data Analyst – By Packt Publishing (2018)

Bharti Motwani, Data Analytics using Python – By Wiley (2020)

Jules S. Damji, Learning Spark: Lightning-Fast Data Analytics, Second Edition – By Shroff/O'Reilly (2020)

Data Science and Machine Learning using Python – 10 August 2022, MGH, 2022